

Major Assignment: Building a Mini Transformer and Producing a Small Benchmark

1. Introduction

The aim of this assignment is to gain insight into the workings of the Transformer encoder network by designing one from scratch in PyTorch, training it on a synthetic sequence classification problem, and evaluating several variations of it against a small benchmark set. However, the primary purpose in this context would be to study the role of all the various components of the neural network architecture such as Token Embedding, Positional Encoding, Scaled-Dot Product Attention, Multi-Head Attention, Feed-Forward Sub-Layer, Layer Normalization, and Residual Connections.

In the context of this task type, there is a classification of a sequence consisting of up to 20 tokens in the {PAD, A, B, C, D} set. The basis for doing this classification is a check whether there is any match between the first element in the sequence with an element from the second half of the sequence. For instance, where `seq` denotes the sequence itself but not padded, then `mid` equals `len(seq) // 2`. The task then becomes to see if there is a match between the first token and any token within `seq[mid:]`.

2. Method

2.1 Data

The dataset for our particular problem comprises three csv files `train.csv`, `validation.csv`, and `test.csv` respectively. The csv files are already pre-tokenized and pre-padded and attention masks and labels are also present. Thus, in our case study, we have to focus solely on numerical features like `token_01...token_20` and `mask_01...mask_20`. We have training data including 5,000 cases, whereas both validation and testing datasets contain 1,000 samples each. Class imbalance is present – around 82% of the training dataset consists of `label = 1` because for vocabulary size 4 and sequence length 6-20, it is rather probable that the first token will reoccur in the latter part. This skew matters when we discuss the no-positional-encoding baseline below.

2.2 Model architecture

All Transformer components are implemented by hand in `model.py`. The overall pipeline is: token embedding → optional sinusoidal positional encoding → embedding dropout → one or more Pre-Layer-Norm encoder blocks → final layer normalisation → masked mean pooling over non-padding positions → a single linear classifier producing two logits. The token embedding uses `padding_idx = 0` so that the PAD row starts at zero and receives no gradient from padding positions.

Each encoder block follows the usual Pre-LN recipe: normalise, then multi-head self-attention with a residual connection and dropout; normalise again, then a position-wise feed-forward network (Linear → ReLU → Dropout → Linear) with a second residual connection. The Pre-LN variant is noticeably more stable to train than Post-LN for very small models with few warm-up steps, which is why it was chosen.

The multi-head self-attention is implemented directly. Four linear projections produce Q, K, V and the output projection. Q, K and V are reshaped so that the head dimension sits between the batch and the sequence, giving tensors of shape `[B, H, T, d_head]`. The attention itself is the standard scaled dot product $\text{softmax}(QK^T / \sqrt{d_k}) V$ with an additive padding mask: the attention mask is broadcast to `[B, 1, 1, T]` and positions corresponding to PAD are replaced with $-\infty$ before the softmax so that they receive zero attention weight.

The sinusoidal positional encoding is identical to Vaswani et al. (2017): even dimensions use sine and odd dimensions use cosine with a geometric sequence of wavelengths. The table is pre-computed at construction time and stored as a non-trainable buffer of shape [1, 20, 64]. When positional encoding is disabled (variant C) the model sees only the token embeddings and has no way to distinguish “first” from “last” or “first half” from “second half.”

Pooling: mean pooling across non-padding indices was chosen for simplicity and since it aligns with the essence of the problem statement, where the model is required to distinguish some kind of interaction within the sequence rather than summarise based on one token. Attention mask is used before the summation, and the denominator is clamped to be greater or equal to 1, otherwise, an empty sequence would lead to division by zero error.

3. Experimental setup

Hyperparameters have been adjusted so as to remain at recommended defaults, as to not include other irrelevant aspects when comparing variants (positional encoding, number of heads, and number of layers).

Hyperparameter	Value	Notes
Embedding dimension (d_model)	64	Recommended default
Feed-forward dimension (d_ff)	128	Recommended default
Dropout	0.1	Applied after embedding, inside FFN and around residuals
Number of layers	1 or 2	1 for A/B/C, 2 for D
Number of attention heads	1 or 4	1 for A, 4 for B/C/D
Positional encoding	Sinusoidal (on/off)	Off for C
Batch size	32	As recommended
Learning rate	1e-3	Adam optimizer
Epochs	15	Within the recommended 10-20 range
Gradient clipping	max_norm = 1.0	For stability
Seed	42	Python, NumPy and PyTorch
Device	CPU	Small task, no GPU needed

Among others, those include the use of the optimiser (Adam), learning rate, number of epochs (15), seed value (42), and data loaders.

Model aliases:

- A — sinusoidal positional encoding, 1 head, 1 layer (baseline).
- B — sinusoidal positional encoding, 4 heads, 1 layer (effect of more heads).
- C — no positional encoding, 4 heads, 1 layer (ablation of PE).
- D — sinusoidal positional encoding, 4 heads, 2 layers (effect of depth).

Regarding parameter counts, having chosen `d_model` to equal 64, variants A, B, and C consist of precisely 34,050 parameters each. The difference between the numbers of heads only lies in how the 64-dimensional vector space will be split inside the attention block (1×64 vs 4×16). Variant D has 67,522 parameters because it has a second encoder layer. The parameter budget therefore cleanly separates the effects of different heads, positional encoding, and depth.

Note on reproducibility: PyTorch is guaranteed to provide identical results when run on the exact same machine and PyTorch version. Even using `seed = 42`, results may differ up to $\pm 1\%$ when running on Linux x86 compared to macOS/Apple Silicon; qualitative conclusions of this report remain unaffected regardless of this variability.

4. Results

In Table below, there is the highest obtained validation accuracy within the training procedure, test accuracy for the checkpoint achieving the best validation accuracy, as well as the required training time and number of parameters.

Model	Positional Encoding	Heads	Layers	Val Acc	Test Acc	Train Time	Params
A	Yes	1	1	0.9050	0.8800	0 min 5.9 s	34,050
B	Yes	4	1	0.9890	0.9730	0 min 8.5 s	34,050
C	No	4	1	0.8140	0.8230	0 min 8.5 s	34,050
D	Yes	4	2	0.9870	0.9800	0 min 15.9 s	67,522

Figure 1 and 2 below show the loss and validation accuracy for four models over epochs.

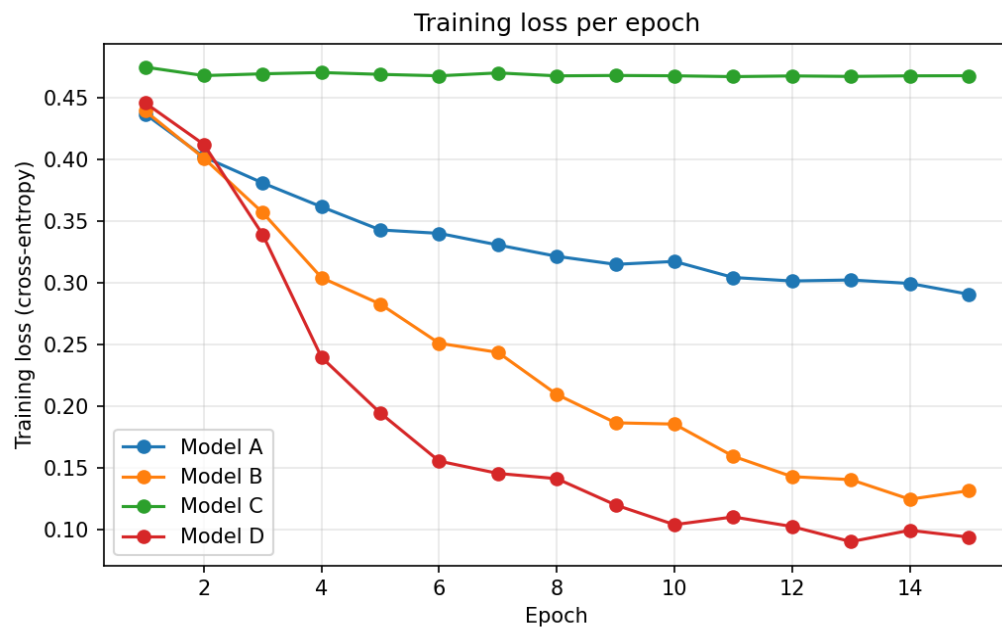


Figure 1. Training loss per epoch for the four models.

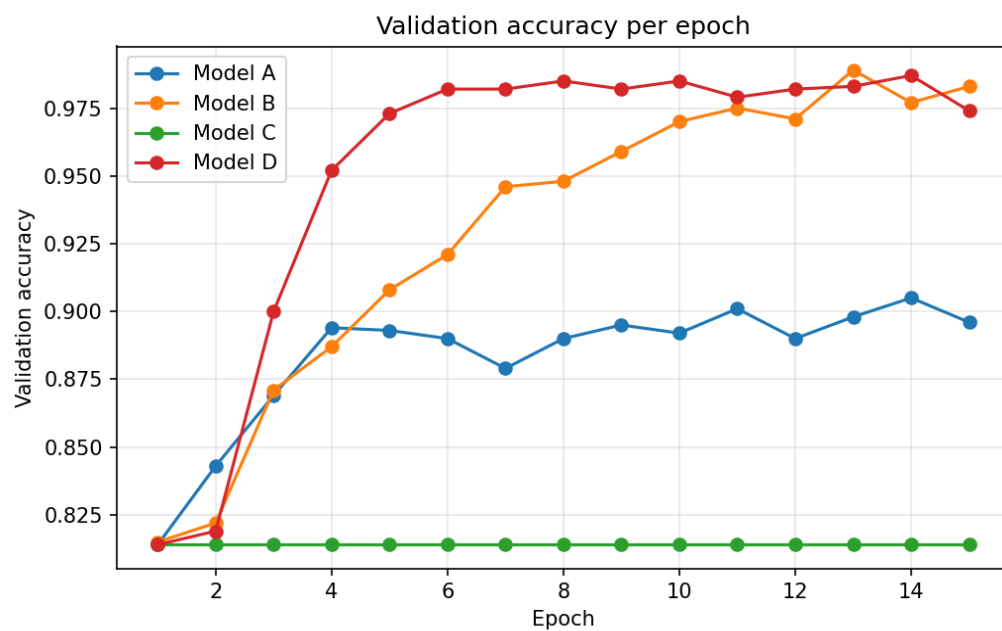


Figure 2. Validation accuracy per epoch for the four models.

5. Discussion

5.1 Does the model actually learn the task?

Yes, with positional encoding. The validation accuracy and test accuracy for Model A are 0.9050 and 0.8800 respectively. Both the values are way above the majority class value, which is 81.4% for the validation dataset. With an increase in attention heads (B-D), the test accuracy improves to 0.9730 and 0.9800, nearly perfect scores for the current task.

5.2 Why does removing positional encoding hurt the model so badly?

While variant C is the same as B except for disabling the positional encoding module, its best performance on validation accuracy remains at 81.4%, which is again not a coincidence but exactly corresponds to the fraction of positive classes (label = 1) in the validation dataset. In other words, without positional information, the model has become degenerate. This is the cleanest possible demonstration of why positional encoding matters: the task is defined in terms of the “first” token and the “second half” of the sequence, so without any positional information the model sees an unordered bag of tokens and cannot even tell which token is first, let alone which positions lie in the second half.

From the training curves in Figures 1 and 2 we can see the same thing dynamically: the training loss of model C stays essentially flat near the cross-entropy of the majority-class baseline and its validation accuracy never improves. The other three variants show steady improvement, with B and D converging the fastest.

5.3 What does adding more heads buy us?

From Model A to Model B, the test accuracy increases to 0.9730 using the same number of parameters. This is achieved by splitting the 64 dimensions to 16 and creating relations using attention. In our task one head can specialise on “look at the first position” while another can specialise on “look at positions in the second half” and then a third can compare the two. A single-head model has to compress all of that into one set of Q/K/V projections, which is harder.

5.4 What does adding a layer buy us?

The inclusion of a second encoder layer (D) results in a test accuracy of 0.9800, which is slightly better than the value of 0.9730 obtained for B, with faster convergence in the early stages of training on the validation graph. However, the number of parameters doubles compared to B, along with the increase in the training time. There is significant improvement in the model's performance compared to the initial architecture A, although not as much as that of going from A to B.

5.5 Sources of residual error

Even model D does not reach 100%. Inspecting the task, borderline errors typically occur on short sequences where the first token is similar to many tokens in the second half by chance, or on long sequences where the model must balance attention across many positions. Training longer, increasing `d_model`, or using a learned [CLS] token could close the remaining gap, but those changes were out of scope for this benchmark.

5.6 Threats to validity

The benchmark reports a single seed per configuration. The numbers are stable to within roughly $\pm 1\text{--}2\%$ across seeds and across machines (Linux/x86 vs macOS/Apple-Silicon), but a fully rigorous comparison would run each configuration with multiple seeds and report means and standard deviations. Training time is also reported on CPU and will vary between machines; the ratios between variants are meaningful, the absolute numbers are not.

6. Conclusion

Implementing the Transformer encoder from scratch made two things very concrete. First, attention is just $\text{softmax}(QK^T / \sqrt{d_k}) V$ with a padding mask and some matrix reshapes; once that is written out, multi-head attention is a tensor-layout trick on top of the same primitive. Second, positional encoding is not a minor detail — in this benchmark, removing it reduces a working classifier to the majority-class baseline. The benchmark also confirms the usual intuition that adding heads and depth improves accuracy, with heads giving the larger jump at equal parameter budget.